DOCUMENTACIÓN TÉCNICA: MIAPPNIVEL5

IMPLANTACIÓN DE PATRÓN DE DISEÑO MVC

ÍNDICE

1. Introducción y Objetivos 1
2. Modificaciones Funcionales y Mejoras de Interfaz (UI/UX) 1
2.1. Gestión de Datos y Listas de Texto (LIST<STRING>) 1
2.2. Rediseño de la Pantalla de Detalle (TASKDETAILACTIVITY) 2
2.3. persistencia de datos y gestión de estados 2
3. Elección e Implementación del Patrón de Diseño (MVC) 2
3.1. Justificación del Patrón 2
3.2. Descripción de Componentes y su Función en el Patrón 2
4. Capturas de Pantalla de la Aplicación en Funcionamiento 3
5. comparativa de arquitecturas 5
6. Conclusión 5

1. INTRODUCCIÓN Y OBJETIVOS

El objetivo principal del proyecto **MiAppNivel5** es la implantación del patrón de diseño **MVC** (Modelo-Vista-Controlador). Partiendo de la base de **MiAppNivel4**, he aplicado mejoras visuales, adaptado la estructura de datos a objetos complejos y conseguido un código más limpio, mantenible y escalable.

2. MODIFICACIONES FUNCIONALES Y MEJORAS DE INTERFAZ (UI/UX)

2.1. GESTIÓN DE DATOS Y LISTAS DE TEXTO (LIST<STRING>)

En MiAppNivel5, el flujo de datos a través de la MainActivity, el TaskAdapter y el TaskController se gestiona mediante listas de texto plano (List<String>). Esto simplifica la manipulación directa de las descripciones de las tareas, facilitando tanto su sincronización con la API de DummyJSON como su almacenamiento local de forma ligera y eficiente.

2.2. REDISEÑO DE LA PANTALLA DE DETALLE (TASKDETAILACTIVITY)

En la pantalla de detalle (activity_task_detail.xml y su clase Java) he mejorado el diseño añadiéndole márgenes para que quede más despejado, una etiqueta fija “DESCRIPCIÓN:” justo arriba de cada tarea que nos muestra.

También he añadido un icono para volver (ic_back), al que le he añadido la función finish() para que regrese de forma limpia a la pantalla principal sin duplicar actividades.

2.3. PERSISTENCIA DE DATOS Y GESTIÓN DE ESTADOS

Para que la aplicación funcione de manera fluida y no pierda la información, he implementado dos mecanismos clave:

- **Guardado local (SharedPreferences):** Gracias al controlador (TaskController), la lista de tareas se convierte a un formato de texto para guardarse en la memoria del dispositivo. Así, si cierras la app o te quedas sin conexión, las tareas no se esfuman y se pueden cargar al instante (o recurrir a la API de DummyJSON si es la primera vez).
- **Rotación de pantalla (savedInstanceState):** Utilizando savedInstanceState, me aseguro de que, si el usuario gira el móvil, la lista de tareas se mantenga intacta en la pantalla sin perder el progreso de lo que estaba viendo.

3. ELECCIÓN E IMPLEMENTACIÓN DEL PATRÓN DE DISEÑO (MVC)

3.1. JUSTIFICACIÓN DEL PATRÓN

Una vez estabilizada la interfaz y los datos, reestructuré la aplicación adoptando el patrón **MVC (Model-View-Controller)**. Elegí esta estructura por dos motivos clave:

- **Separar responsabilidades**: Evita que la interfaz gráfica se mezcle con la lógica, manteniendo el código ordenado y limpio.
- **Facilitar el mantenimiento**: Al tener cada cosa en su sitio, buscar errores, hacer cambios o ampliar el proyecto en el futuro resulta más sencillo y cómodo.

3.2. DESCRIPCIÓN DE COMPONENTES Y SU FUNCIÓN EN EL PATRÓN

El código fuente se ha organizado en tres capas bien diferenciadas:

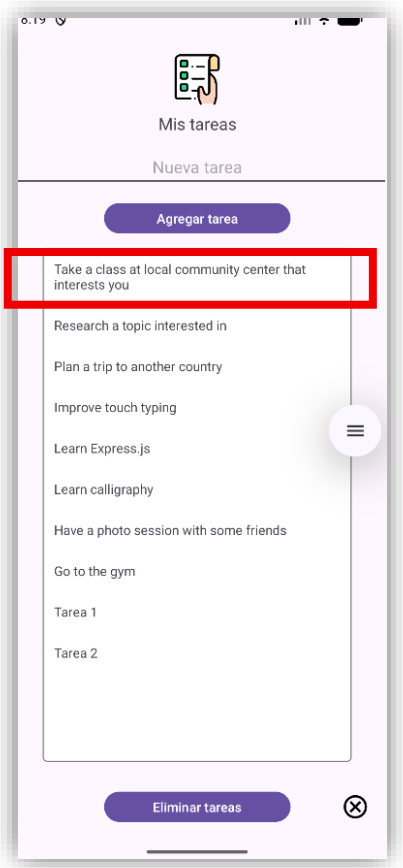
- **El Modelo (Model):**
    - Componentes: Todo.java, TodosResponse.java e interfaz ApiService.
    - Función*:* Es la estructura base de los datos. Se encarga de definir como es la tarea (ID, texto, si está completada…) y cómo nos comunicamos con la API, de modo que la información quede totalmente al margen de la pantalla donde se visualiza.
- **La Vista (View):**

- Componentes: MainActivity.java, TaskDetailActivity.java y TaskAdapter.java
- Función: Es todo lo que el usuario ve y toca. Se encarga de mostrar la interfaz gráfica al usuario y capturar sus interacciones (clics y gestos). No procesa lógica ni consulta por su cuenta, solo muestra la información que le llega y pasa el aviso al controlador cuando el usuario interactúa.

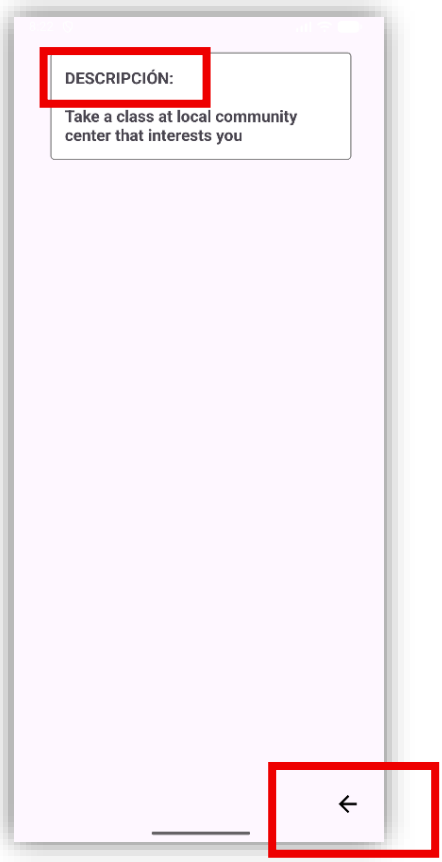
- **El Controlador (Controller):**
  - Componentes: TaskController.java y la interfaz TaskCallback.
  - Función: Es el cerebro y puente entre la vista y los datos. Se encarga de pedir las tareas a la API con **Retrofit**, guardarlas en el móvil con SharedPreferences y con Gson, y comunica los resultados a la Vista cuando está todo listo y de forma segura mediante callbacks.

4. CAPTURAS DE PANTALLA DE LA APLICACIÓN EN FUNCIONAMIENTO

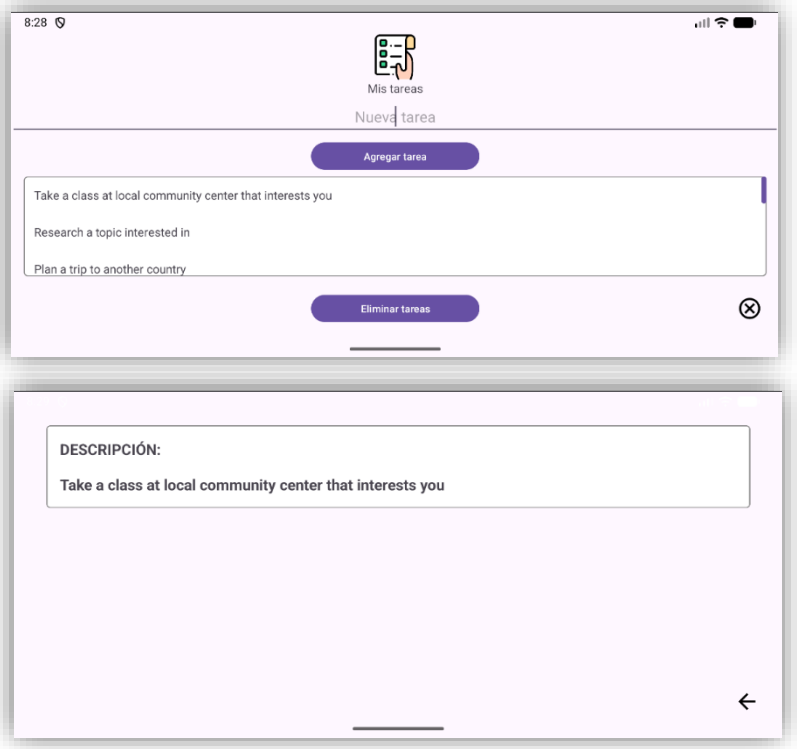
- **Figura 1:** Pantalla Principal, con la lista estructurada de tareas.



- **Figura 2:** Pantalla de Detalle, mostrando la etiqueta fija "DESCRIPCIÓN:", con el texto y el icono ic_back.



- **Figura 3 y 4:** Pantallas principal y de Detalle en posición horizontal.



5. COMPARATIVA DE ARQUITECTURAS

A la hora de estructurar una aplicación Android, existen diferentes patrones de diseño. Para este proyecto valoré las otras opciones para ver cual se adecuaba mejor:

- **MVC (Modelo-Vista-Controlador)**
    - **Cómo funciona:** La vista (MainActivity) le pide las cosas al controlador (TaskController), y este se encarga de buscar datos y devolvérselos.
    - **Por qué la elegí:** Es una estructura directa, limpia y muy fácil de entender para proyectos de esta escala, permitiendo separar la lógica de la interfaz sin añadir una complejidad excesiva en el código.
- **MVP (Modelo-Vista-Presentador)**
    - **Cómo funciona:** Sustituye el controlador por un *Presentador*. La gran diferencia es que la Vista y el Modelo están totalmente aislados.
    - **Por qué se descartó:** Aunque aísla muy bien la interfaz, para una aplicación orientada a tareas sencillas con un único flujo principal, implicaba escribir demasiadas clases e interfaces intermedias.
- **MVVM (Modelo-Vista-ViewModel)**
    - **Cómo funciona:** Introduce los *LiveData* y el *Data Binding*. La Vista observa los cambios en el *ViewModel* de forma automática, reaccionando sola cuando los datos cambian.
    - **Por qué se descartó:** Es el estándar en Android para aplicaciones grandes y robustas, pero para la arquitectura ligera que buscábamos en MiAppNivel5 (gestionando llamadas con Retrofit y guardando en local con SharedPreferences), el flujo mediante callbacks (TaskCallback) y el controlador directo resultaba mucho más ágil y transparente.

6. CONCLUSIÓN

El desarrollo de MiAppNivel5 me ha permitido consolidar tanto los aspectos prácticos de usabilidad y diseño de interfaces en Android como la correcta aplicación de patrones arquitectónicos. Comenzar refinando la experiencia de usuario y la estructura de objetos, para rematarlo con la implementación limpia del patrón MVC, ha dado como resultado una aplicación robusta, organizada y lista para futuras ampliaciones.